

HANDWRITTEN ALPHABET DETECTOR

Deep Learning Project

Image Classification Using Convolutional Neural Networks (CNN)

Tools Used:

Python | TensorFlow | Keras | NumPy | Scikit-learn | Pillow | Tkinter | Matplotlib

Dataset:

Handwritten English Alphabets (A–Z)

Prepared by:

John Joshua J

Date:

September 2026

ABSTRACT

Handwritten character recognition is an important application of Artificial Intelligence, Machine Learning, and Computer Vision. This project presents a Handwritten Alphabet Detector developed using a Convolutional Neural Network (CNN) to recognize handwritten English alphabets from A to Z. The system is designed to classify a handwritten character by learning visual patterns and features from a dataset of handwritten alphabet images.

The project uses Python, TensorFlow, Keras, NumPy, Scikit-learn, Pillow, Matplotlib, and Tkinter for model development, image processing, evaluation, visualization, and graphical user interface implementation. The input images are converted to grayscale, the character region is identified and cropped, padding is applied, the character is resized to 28×28 pixels, and pixel values are normalized before being provided to the CNN.

The CNN consists of convolutional, batch normalization, and max-pooling layers followed by fully connected layers and a Softmax output layer for classification into 26 alphabet classes. The trained model achieved a test accuracy of 96.15% on 286 test images. During individual prediction testing, the character A was correctly recognized with a confidence of 99.82%.

A Tkinter-based graphical user interface was also developed. It allows users to draw a handwritten alphabet using a mouse or upload an image. The application preprocesses the input and uses the trained CNN model to predict the corresponding alphabet and display its confidence score.

The project demonstrates an end-to-end deep learning workflow involving dataset preparation, image preprocessing, CNN model development, model training, performance evaluation, prediction on unseen images, model saving, and interactive GUI-based prediction.

1. INTRODUCTION

Handwritten character recognition is an important application of Artificial Intelligence, Machine Learning, and Computer Vision. It involves identifying handwritten characters from images and converting them into corresponding digital characters.

The objective of this project is to develop a Handwritten Alphabet Detector using a Convolutional Neural Network (CNN). The system is designed to recognize handwritten English alphabets from A to Z. A CNN is trained using handwritten character images and learns important visual features such as edges, curves, shapes, and patterns.

The project uses Python and TensorFlow/Keras for developing and training the deep learning model. Image preprocessing techniques are applied to convert input images into a consistent format before classification. The images are converted to grayscale, the character region is detected and cropped, padding is applied, the images are resized to 28×28 pixels, and pixel values are normalized.

A graphical user interface is developed using Tkinter, allowing users to draw a handwritten alphabet using the mouse or upload an existing image. The trained CNN model processes the input and predicts the corresponding alphabet along with its confidence score.

The developed model achieved a test accuracy of 96.15% on the available test dataset containing 286 images across 26 alphabet classes. The project demonstrates the practical application of Convolutional Neural Networks for handwritten image classification and their integration into an interactive desktop application.

2. PROBLEM STATEMENT

Recognizing handwritten English alphabets automatically is a challenging image classification problem because handwritten characters can vary significantly in shape, size, thickness, position, and writing style. The same alphabet may appear differently when written by different individuals.

The problem addressed in this project is to develop an automated system capable of identifying a handwritten English alphabet from an input image and classifying it into one of the 26 alphabet classes, A to Z.

To solve this problem, a Convolutional Neural Network (CNN) is trained using handwritten English alphabet images. The images are preprocessed by converting them to grayscale, identifying the character region, cropping unnecessary background areas, adding padding, resizing the character to 28×28 pixels, and normalizing pixel values.

The trained CNN is integrated with a Tkinter graphical user interface. Users can draw a character or upload an image, after which the application preprocesses the input and predicts the corresponding alphabet along with its confidence score.

The overall problem therefore involves developing an end-to-end system combining image preprocessing, CNN-based classification, model evaluation, and interactive alphabet prediction.

3. OBJECTIVES

The main objective of this project is to develop a deep learning-based system for recognizing handwritten English alphabets using a Convolutional Neural Network.

The specific objectives are:

1. To collect and prepare handwritten alphabet image data containing English alphabets from A to Z.
2. To preprocess input images by converting them to grayscale, cropping unnecessary areas, resizing them to a standard dimension, and normalizing pixel values.

3. To design and develop a Convolutional Neural Network capable of learning visual features from handwritten alphabet images.
4. To train the CNN model using the prepared dataset and optimize its ability to classify handwritten characters.
5. To evaluate the trained model using test accuracy, loss, classification report, and confusion matrix.
6. To develop a graphical user interface using Tkinter that allows users to draw handwritten alphabets using a mouse.
7. To provide image-upload functionality so users can test the trained model using existing alphabet images.
8. To display the predicted alphabet and confidence score after classification.
9. To save the trained CNN model so that it can be reused for future predictions without retraining.
10. To demonstrate the practical application of deep learning and computer vision through an interactive handwritten alphabet recognition system.

4. DATASET DESCRIPTION

The dataset used in this project consists of handwritten English character images representing the 26 English alphabets from A to Z. The images are used to train, validate, and test the Convolutional Neural Network.

The dataset contains handwritten samples with variations in writing style, shape, size, thickness, and positioning. These variations help the CNN learn features that are useful for recognizing different representations of the same alphabet.

4.1 Dataset Classes

The project uses 26 alphabet classes:

A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z

4.2 Dataset Organization

The dataset is organized into separate training and testing directories. Each directory contains 26 class folders corresponding to the English alphabets A to Z. Each class folder contains images belonging to that alphabet.

archive/

└─ handwritten-english-characters-and-digits/

└─ combined_folder/

└─ train/

└─ A_caps/

└─ B_caps/

└─ ...

└─ Z_caps/

|

└─ test/

└─ A_caps/

└─ B_caps/

└─ ...

└─ Z_caps/

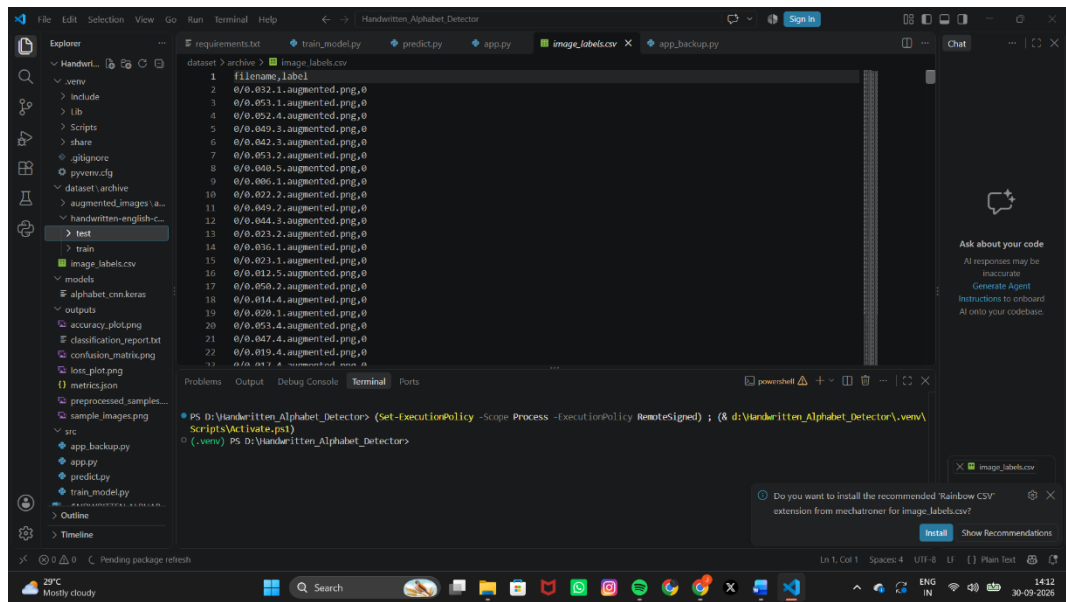


Figure 4.1: Dataset folder structure

4.3 Dataset Preparation

The images are loaded from their respective class directories. The directory names are used to associate each image with its corresponding alphabet label.

The images are subsequently processed into a consistent format suitable for CNN training.

4.4 Training and Testing Data

The training dataset contains 1,144 images, with 44 images for each of the 26 alphabet classes. During training, the training data were divided into 972 training samples and 172 validation samples. The separate test dataset contains 286 images, with 11 images for each alphabet class.

4.5 Data Augmentation

Data augmentation can be used to introduce controlled variations in training images, such as small rotations, shifts, and zoom transformations.

These transformations can help the model learn more general patterns and improve its ability to handle variations in handwriting.

5. DATA PREPROCESSING AND PREPARATION

Data preprocessing is an important stage of the project because raw handwritten images may differ in size, background, position, and appearance. The preprocessing pipeline converts these images into a consistent representation suitable for the CNN.

The preprocessing workflow is:

Input Image → Grayscale → Character Detection → Cropping → Padding → Resize → Normalize → CNN

5.1 Image Loading

The input image is loaded using the Pillow image-processing library. The same general preprocessing approach is applied to images obtained from the dataset and images supplied through the GUI.

5.2 Grayscale Conversion

The image is converted into grayscale so that it contains a single intensity channel instead of separate red, green, and blue channels.

This reduces unnecessary color information while preserving the shape and structure of the handwritten character.

5.3 Character Detection

The system identifies the region containing the handwritten character by examining the image pixels.

This allows unnecessary empty background areas to be removed before classification.

5.4 Cropping

After detecting the character region, unnecessary surrounding background is removed.

Cropping helps the CNN concentrate on the character itself rather than large empty portions of the image.

5.5 Padding

Padding is added around the cropped character to prevent it from touching the image boundaries.

This provides a consistent representation before resizing.

5.6 Image Resizing

The processed character is resized to:

28×28 pixels

This matches the input dimensions expected by the trained CNN.

5.7 Pixel Normalization

Pixel values are normalized to a range between 0 and 1.

The normalization formula is:

Normalized Pixel Value = Pixel Value / 255

For example, a pixel value of 255 becomes 1, while a pixel value of 0 remains 0.

5.8 Image Inversion

Image inversion is applied when necessary so that the character representation matches the format used during CNN training.

Maintaining the same representation during training and prediction is important for consistent model performance.

5.9 Final Input Format

After preprocessing, each image is represented as:

$$28 \times 28 \times 1$$

where:

- 28 = image height
- 28 = image width
- 1 = grayscale channel

The processed image is then passed to the CNN.

6. EXPLORATORY DATA ANALYSIS AND SAMPLE VISUALIZATION

Exploratory Data Analysis (EDA) is performed to understand the characteristics of the dataset before model training.

6.1 Purpose of EDA

The main objectives are:

- To understand the organization of the dataset.
- To identify the available alphabet classes.

- To inspect sample handwritten images.
- To verify that images are loaded correctly.
- To observe variations in handwriting.
- To identify possible preprocessing requirements.

6.2 Sample Image Visualization

Sample images from different alphabet classes are visualized using Python and Matplotlib.

The visualization provides a visual confirmation that the dataset contains handwritten characters corresponding to their class labels.

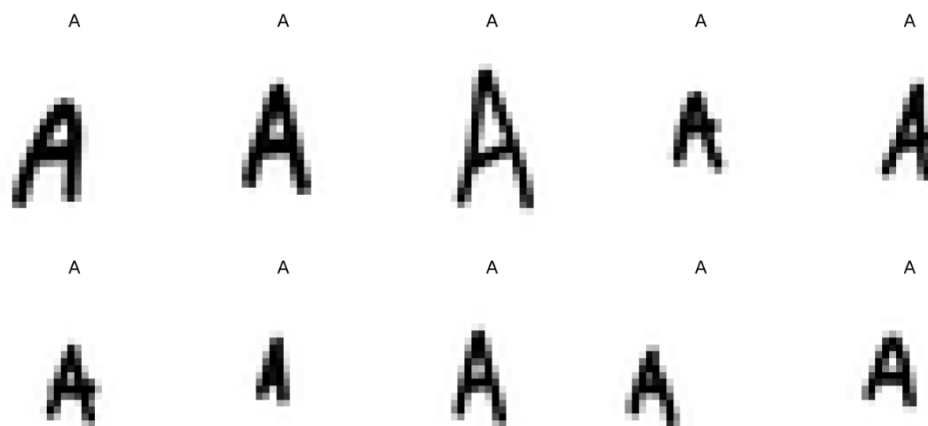


Figure 6.1: Sample handwritten alphabet images from the dataset

6.3 Image Characteristics

The images contain handwritten character patterns represented through pixel values.

Different samples may vary in:

- Character shape
- Thickness

- Size
- Position
- Writing style
- Orientation

These variations make CNN-based feature learning useful for the classification task.

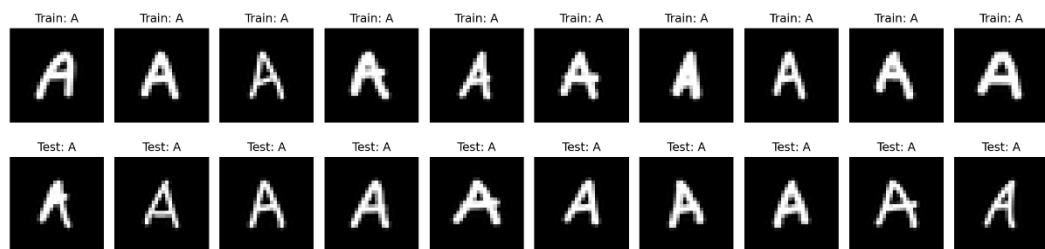


Figure 6.2: Preprocessed handwritten alphabet samples

6.4 Class Distribution

The number of samples belonging to each class can be examined to determine whether the dataset is reasonably distributed across the alphabet classes.

Class distribution analysis can also help identify classes with comparatively fewer samples.

6.5 Importance of Visualization

Visualization helps verify:

1. The images contain handwritten characters.
2. Images correspond to their labels.
3. Images are loaded successfully.
4. Different handwriting styles are present.
5. The dataset is suitable for CNN-based classification.

7. TECHNOLOGIES AND TOOLS USED

7.1 Python

Python is the primary programming language used for data preprocessing, CNN development, training, evaluation, prediction, and GUI development.

7.2 TensorFlow and Keras

TensorFlow and Keras are used to construct, train, evaluate, and save the CNN model.

7.3 NumPy

NumPy is used for numerical operations and processing image data as arrays.

7.4 Scikit-learn

Scikit-learn is used for model evaluation, including generation of the classification report and confusion matrix.

7.5 Pillow

Pillow is used for image loading, conversion, cropping, resizing, saving, and preprocessing.

7.6 Matplotlib

Matplotlib is used to visualize sample images and generate training and validation accuracy and loss graphs.

7.7 Tkinter

Tkinter is used to develop the graphical user interface, including the drawing canvas, buttons, prediction display, and image-upload functionality.

7.8 Visual Studio Code

Visual Studio Code is used as the development environment for writing, organizing, executing, and debugging the project code.

7.9 Windows

The project was developed and tested in a Windows environment using a Python virtual environment to manage project dependencies.

8. SYSTEM ARCHITECTURE

The Handwritten Alphabet Detector follows an end-to-end image classification workflow.

The overall system is:

User Input → Image Preprocessing → CNN Model → Feature Extraction → Classification → Prediction → Confidence Score → GUI Output

8.1 System Workflow

Step 1: User Input

The user provides a handwritten alphabet by:

- Drawing it on the Tkinter canvas, or
- Uploading an existing image.

Step 2: Image Preprocessing

The input is:

- Converted to grayscale.
- Processed to identify the character.
- Cropped.
- Padded.
- Resized to 28×28 pixels.
- Normalized.
- Prepared as a $28 \times 28 \times 1$ tensor.

Step 3: CNN Model

The preprocessed image is passed to the trained CNN.

Step 4: Feature Extraction

Convolutional and pooling layers extract meaningful features such as edges, curves, and shapes.

Step 5: Classification

The extracted features are passed through fully connected layers.

The final layer contains 26 classes representing A–Z.

Step 6: Prediction

The model calculates probabilities for all 26 classes.

The class with the highest probability is selected as the predicted alphabet.

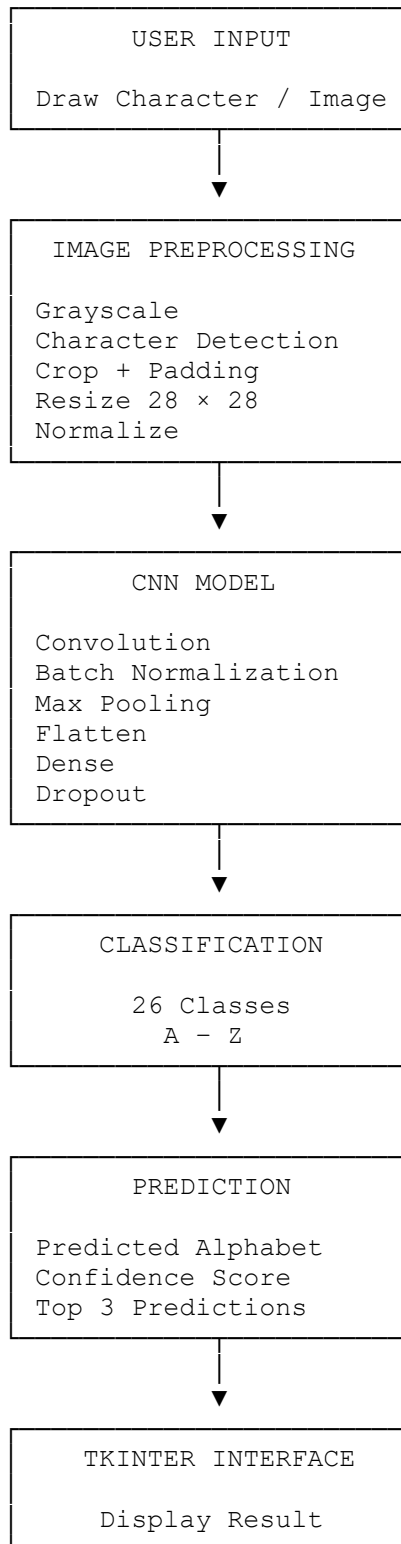
Step 7: Confidence Score

The probability associated with the predicted class is displayed as the confidence score.

Step 8: Output

The prediction and confidence score are displayed through the Tkinter interface.

8.2 Architecture Diagram



9. CNN MODEL ARCHITECTURE

A Convolutional Neural Network is used as the main deep learning model for handwritten alphabet classification.

CNNs are suitable for image classification because they can automatically learn visual features from images.

Input: $28 \times 28 \times 1$

Conv2D – 32 filters

Batch Normalization

Max Pooling

Conv2D – 64 filters

Batch Normalization

Max Pooling

Conv2D – 128 filters

Batch Normalization

Max Pooling

Flatten

Dense – 128 neurons

Dropout

Dense – 26 neurons

Softmax

The model contains 244,506 total parameters, of which 244,058 are trainable and 448 are non-trainable.

9.1 Input Layer

The CNN accepts a grayscale image with dimensions:

$$28 \times 28 \times 1$$

The final dimension represents the single grayscale channel.

9.2 Convolutional Layers

Convolutional layers apply filters to the image to learn important visual patterns.

The model learns features such as:

- Edges
- Lines
- Curves
- Corners
- Character structures

Multiple convolutional layers allow the network to learn increasingly complex features.

9.3 Batch Normalization

Batch normalization is used to normalize intermediate activations during training.

It can help stabilize the learning process and improve training efficiency.

9.4 Max Pooling

Max-pooling layers reduce the spatial dimensions of feature maps while retaining important information.

This reduces computational requirements and allows the network to focus on prominent features.

9.5 Flatten Layer

The feature maps generated by the convolutional and pooling layers are converted into a one-dimensional vector using the Flatten layer.

9.6 Dense Layer

The extracted features are passed to fully connected layers.

The main dense layer contains 128 neurons according to the model configuration used in the project.

9.7 Dropout Layer

Dropout is used to reduce overfitting by temporarily disabling a portion of neurons during training.

9.8 Output Layer

The final layer contains 26 neurons, corresponding to:

A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z

A Softmax activation function converts the outputs into probabilities.

The class with the highest probability becomes the predicted alphabet.

9.9 Model Flow

Input Image



$28 \times 28 \times 1$



Convolution



Batch Normalization



Max Pooling



Convolution



Batch Normalization



Max Pooling



Flatten



Dense – 128 Neurons



Dropout



Dense – 26 Neurons



Softmax



Predicted Alphabet

```
(.venv) PS D:\Handwritten_Alphabet_Detector> python -c "import tensorflow as tf; model=tf.keras.models.load_model('models/alphabet_cnn.keras'); model.summary()"

```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	320
batch_normalization (BatchNormalization)	(None, 28, 28, 32)	128
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 14, 14, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_2 (Conv2D)	(None, 7, 7, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 7, 7, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 3, 3, 128)	0
flatten (Flatten)	(None, 1152)	0
dense (Dense)	(None, 128)	147,584
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 26)	3,354

```

Total params: 732,624 (2.79 MB)
Trainable params: 244,058 (953.35 KB)
Non-trainable params: 448 (1.75 KB)
Optimizer params: 488,118 (1.86 MB)
(.venv) PS D:\Handwritten_Alphabet_Detector>

```

Figure 9.1: CNN model architecture summary

10. MODEL COMPILATION AND TRAINING

After preprocessing, the dataset is used to train the CNN model.

10.1 Training Process

The training process consists of:

1. Loading the handwritten alphabet dataset.
2. Organizing images according to alphabet classes.
3. Preprocessing the images.
4. Dividing the available data into training and validation data.
5. Passing training images through the CNN.
6. Calculating classification loss.
7. Updating trainable parameters using the optimizer.
8. Monitoring training and validation performance.

9. Evaluating the model using the test dataset.

10. Saving the trained model.

10.2 Model Configuration

Component	Configuration
Input Size	$28 \times 28 \times 1$
Number of Classes	26
Output Classes	A–Z
Model Type	Convolutional Neural Network
Framework	TensorFlow / Keras
Loss Function	Sparse Categorical Cross-Entropy
Output Activation	Softmax
Main Evaluation Metric	Accuracy

10.3 Training Performance

The reported training results are:

Metric	Result
Training Accuracy at final displayed epoch	99.90%
Validation Accuracy at final displayed epoch	94.77%
Test Accuracy	96.15%
Test Loss	0.1627
Test Images	286
Completed Epochs	28

The training accuracy increased as the model learned features from the training images. Validation accuracy was monitored to assess how well the model generalized to data not directly used for parameter updates.

10.4 Model Saving

After training and evaluation, the trained model was saved in Keras format.

Model path:

models\alphabet_cnn.keras

Saving the model allows the application to load the trained CNN later without repeating the training process.

10.5 Training and Validation Graphs

The project generates:

outputs/

└─ accuracy_plot.png

└─ loss_plot.png

The accuracy graph shows training and validation accuracy over the training process.

The loss graph shows training and validation loss over the training process.

These graphs provide visual evidence of model learning and generalization.

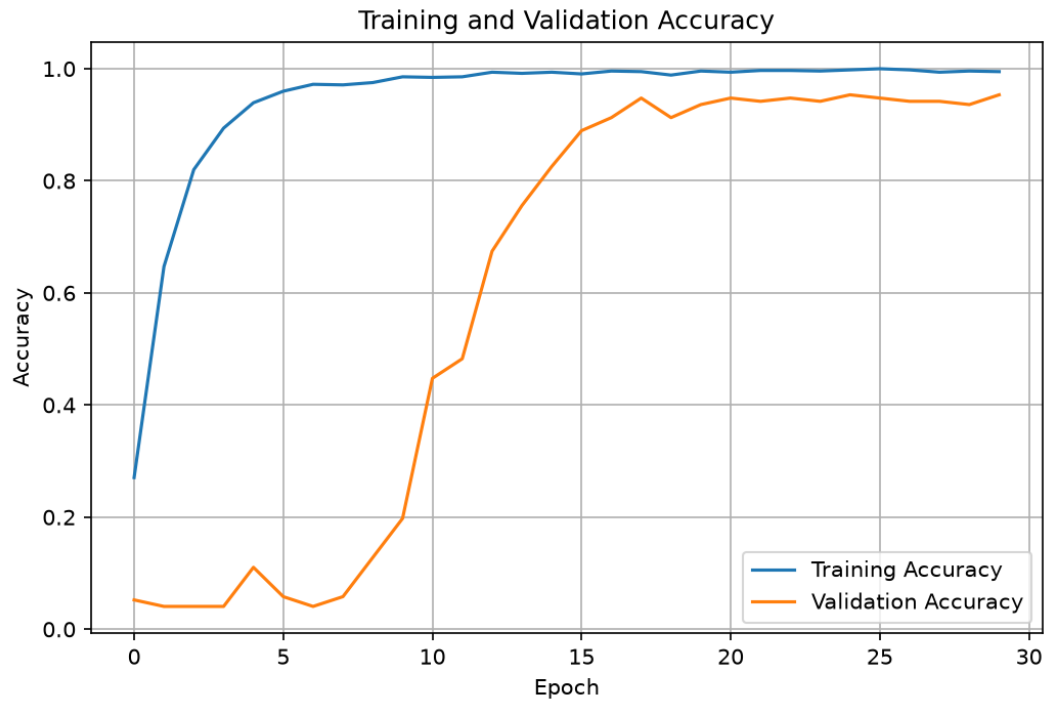


Figure 10.1: Training and validation accuracy

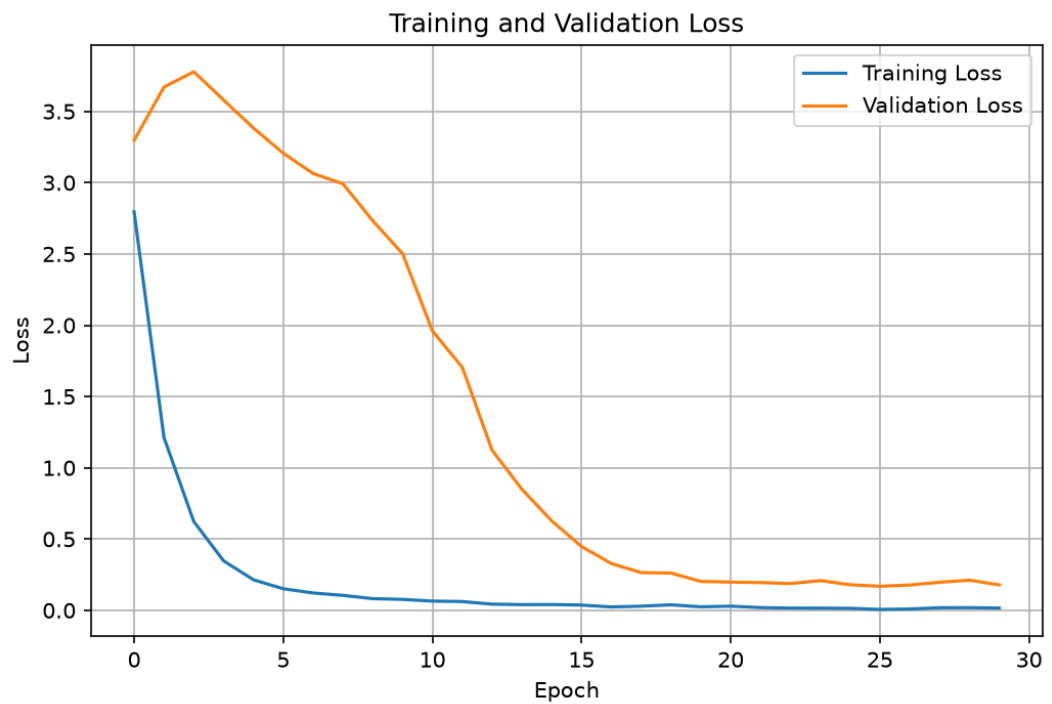


Figure 10.2: Training and validation loss

11. MODEL EVALUATION AND RESULTS

After training, the CNN was evaluated using the test dataset.

11.1 Overall Performance

Evaluation Metric	Result
Number of Classes	26
Test Images	286
Training Accuracy at final displayed epoch	99.90%
Validation Accuracy at final displayed epoch	94.77%
Test Accuracy	96.15%
Test Loss	0.1627

The reported test accuracy of 96.15% indicates that the model correctly classified the majority of the images in the test dataset.

11.2 Classification Report

A classification report was generated using the model's test predictions.

The report provides:

- Precision
- Recall
- F1-score
- Support

for each alphabet class.

The classification report shows the precision, recall, F1-score, and support for each of the 26 alphabet classes. The overall test accuracy was **96.15%** across 286 test images

	precision	recall	f1-score	support
A	1.00	1.00	1.00	11
B	1.00	0.91	0.95	11
C	1.00	1.00	1.00	11
D	1.00	1.00	1.00	11
E	0.91	0.91	0.91	11
F	0.77	0.91	0.83	11
G	0.85	1.00	0.92	11
H	0.85	1.00	0.92	11
I	1.00	0.91	0.95	11
J	0.85	1.00	0.92	11
K	1.00	0.91	0.95	11
L	1.00	0.91	0.95	11
M	1.00	0.91	0.95	11
N	1.00	1.00	1.00	11
O	1.00	1.00	1.00	11
P	1.00	0.73	0.84	11
Q	1.00	1.00	1.00	11
R	1.00	0.91	0.95	11
S	1.00	1.00	1.00	11
T	1.00	0.91	0.95	11
U	1.00	1.00	1.00	11
V	1.00	0.91	0.95	11
W	0.92	1.00	0.96	11
X	0.92	1.00	0.96	11
Y	1.00	1.00	1.00	11
Z	0.92	1.00	0.96	11
accuracy			0.95	286
macro avg	0.96	0.95	0.95	286
weighted avg	0.96	0.95	0.95	286

Figure 11.1: Classification report for the test dataset

11.3 Confusion Matrix

A confusion matrix was generated to visualize the classification results for all 26 alphabet classes.

It helps identify correctly classified samples and classes that may be confused with visually similar characters.

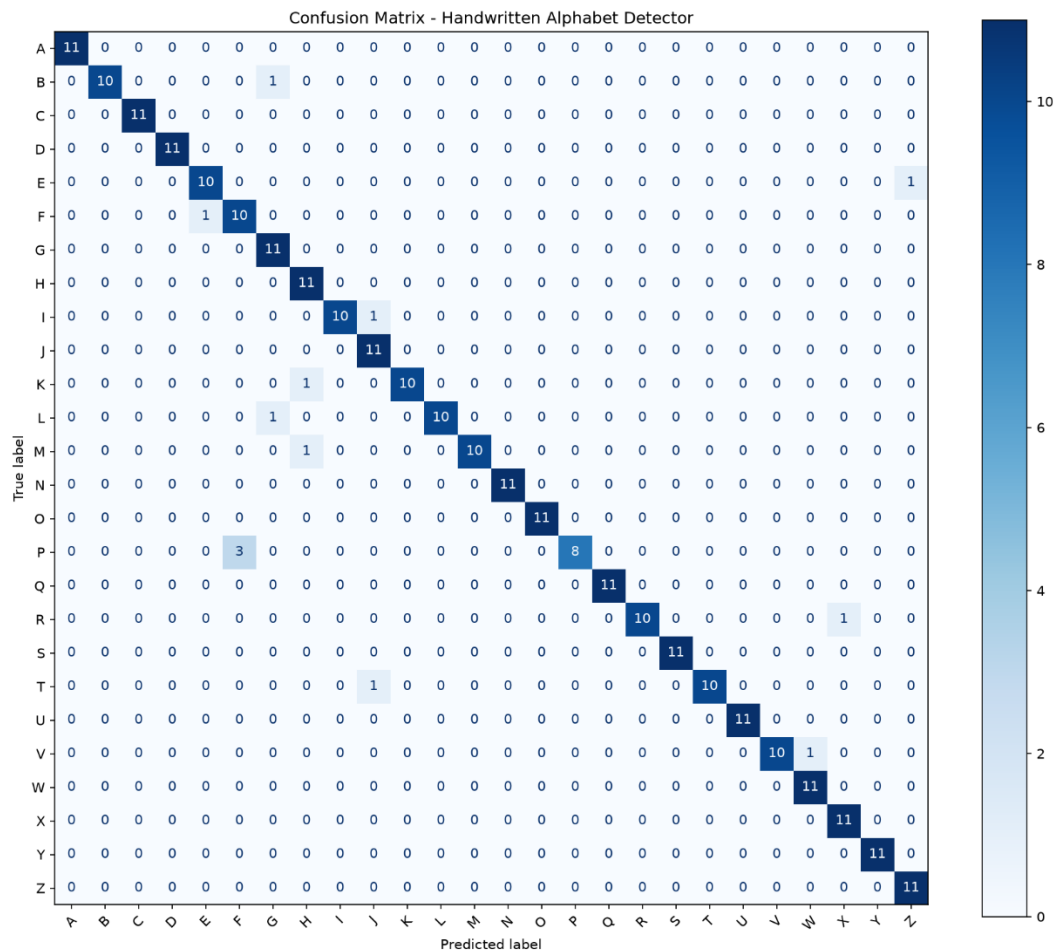


Figure 11.2: Confusion matrix for alphabet classification

11.4 Sample Prediction

The trained model was tested using an individual image:

The reported prediction was:

Predicted Letter: A

Confidence: 99.82%

This demonstrates that the trained CNN can process an unseen input image and produce an alphabet prediction with a confidence score.

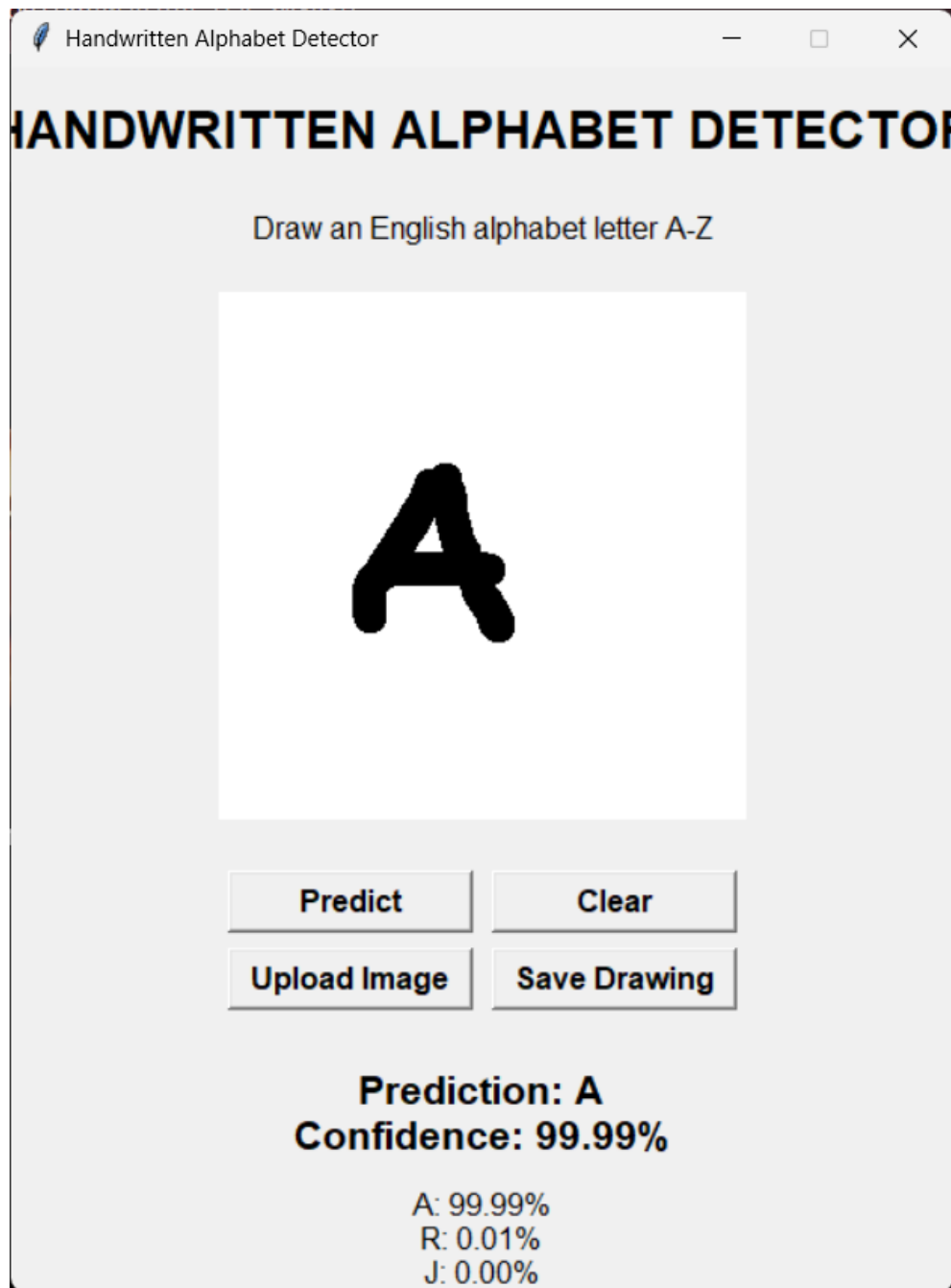


Figure 11.3: Sample alphabet prediction result

11.5 Result Files

The project generates several evaluation files:

outputs/

└─ preprocessed_samples.png

└─ accuracy_plot.png

└─ loss_plot.png
└─ confusion_matrix.png
└─ classification_report.txt
└─ metrics.json

These files provide numerical and visual evidence of the model's performance.

12. TKINTER GRAPHICAL USER INTERFACE

A graphical user interface was developed using Python Tkinter to provide an interactive way to use the trained model.

The application allows users to draw an alphabet on a canvas or upload an existing image.

12.1 Main Features

Draw Character

The user can draw a handwritten alphabet directly on the canvas using the mouse.

Predict

The Predict button processes the drawing and passes it to the trained CNN.

The application displays:

- Predicted alphabet
- Confidence percentage
- Top three predictions

Clear

The Clear button removes the current drawing and resets the prediction display.

Upload Image

The user can select an existing handwritten alphabet image from the computer.

Save Drawing

The drawing can be saved as an image file for future use.

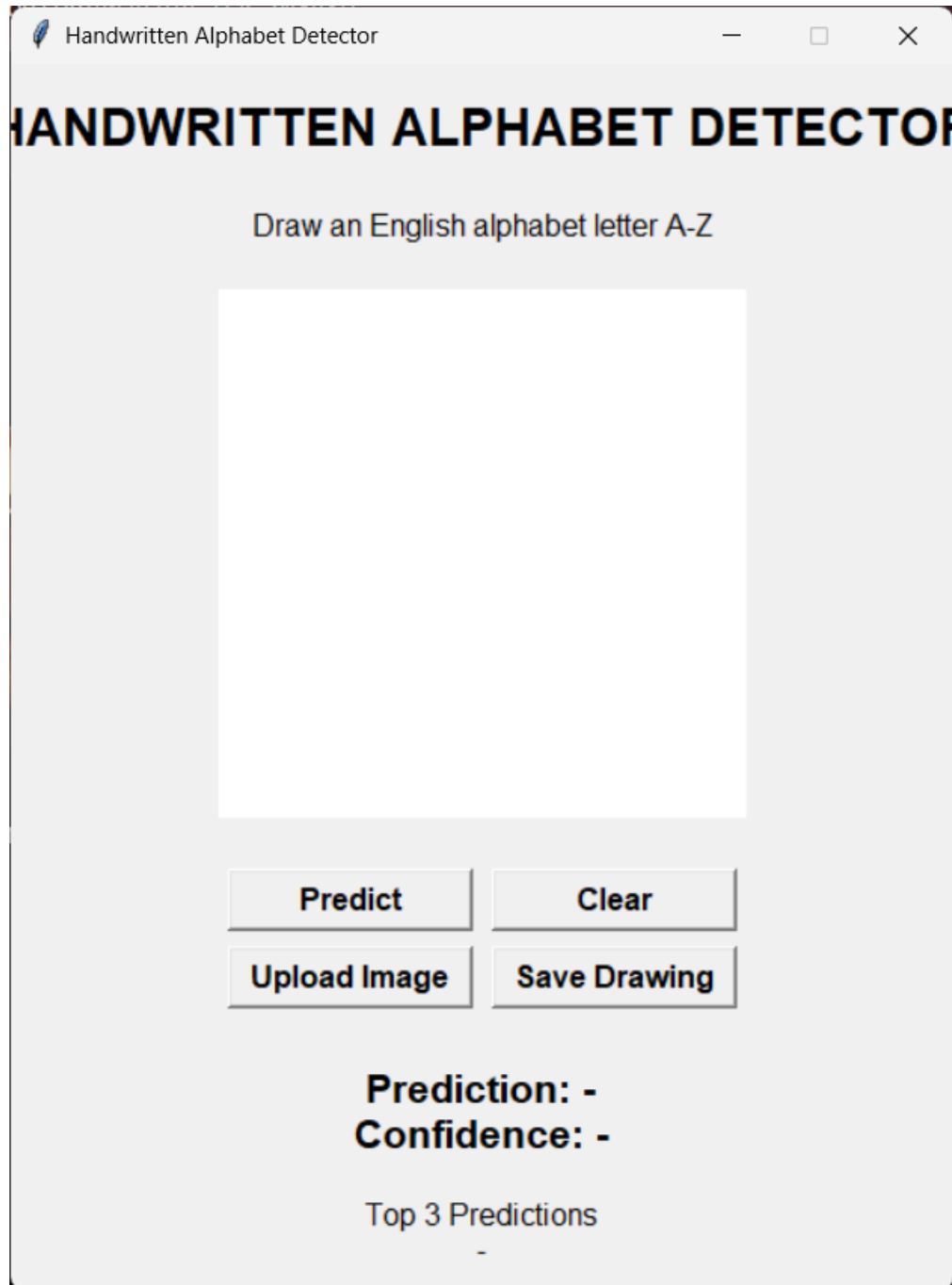
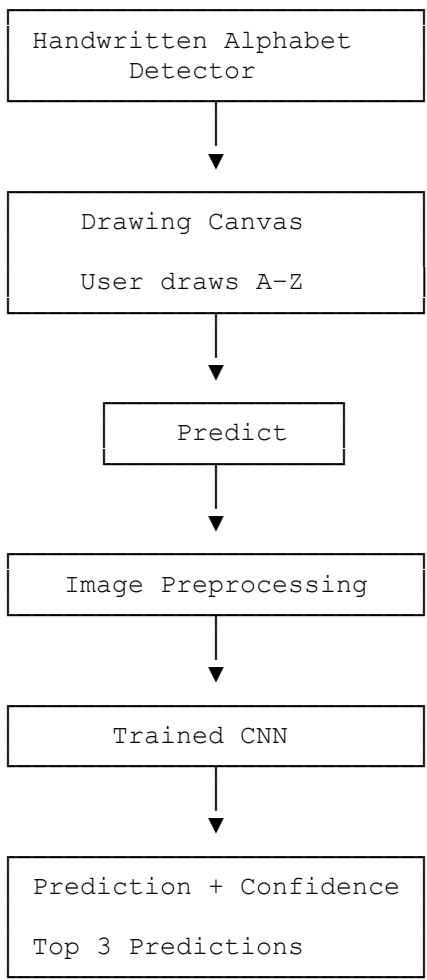


Figure 12.1: Tkinter graphical user interface

12.2 Application Workflow



12.3 Application Components

Component	Purpose
Drawing Canvas	Allows the user to draw a character
Predict Button	Performs alphabet classification
Clear Button	Clears the current drawing
Upload Image	Loads an existing image
Save Drawing	Saves the drawn character
Result Display	Shows the predicted alphabet

Confidence Display	Shows prediction confidence
--------------------	-----------------------------

Top 3 Predictions	Shows highest-probability classes
-------------------	-----------------------------------

12.4 Model Integration

The Tkinter application loads the trained model from:

models\alphabet_cnn.keras

When the user selects Predict, the application:

1. Captures the drawing or uploaded image.
2. Converts it to grayscale.
3. Detects the character.
4. Crops unnecessary background.
5. Adds padding.
6. Resizes the image to 28×28 .
7. Normalizes pixel values.
8. Passes the image to the CNN.
9. Obtains probabilities for all 26 classes.
10. Selects the highest-probability class.
11. Displays the predicted alphabet and confidence score.

12.5 GUI Prediction Result

During testing, the sample alphabet image was correctly classified as:

Predicted Letter: A

Confidence: 99.82%

13. ADVANTAGES AND LIMITATIONS

13.1 Advantages

The Handwritten Alphabet Detector provides the following advantages:

1. Automated Recognition

The system automatically identifies handwritten English alphabets.

2. Deep Learning-Based Classification

The CNN automatically learns relevant image features.

3. Good Classification Performance

The reported model achieved 96.15% test accuracy on the available test dataset.

4. Interactive User Interface

Users can draw characters directly using a mouse.

5. Image Upload Support

Existing handwritten alphabet images can also be tested.

6. Confidence Score

The application displays the probability associated with the predicted alphabet.

7. Top 3 Predictions

The application provides additional prediction possibilities.

8. Reusable Trained Model

The trained model is saved in .keras format and can be reused.

9. Simple User Interface

The application provides straightforward controls such as Predict, Clear, Upload Image, and Save Drawing.

10. Practical CNN Application

The project demonstrates how deep learning can solve a practical image classification problem.

13.2 Limitations

1. Limited to English Alphabets

The current model recognizes only the 26 English alphabets.

2. Dependence on Training Data

Performance depends on the quality and diversity of the training dataset.

3. Handwriting Variation

Unusual handwriting styles may be more difficult to classify.

4. Similar Character Shapes

Visually similar handwritten characters may sometimes be confused.

5. Single Character Recognition

The current system is designed primarily for individual alphabet recognition rather than complete words or sentences.

6. Image Quality

Very unclear, noisy, or poorly positioned images may reduce prediction performance.

7. Hardware Requirements

CNN training can require significant computational resources.

8. Generalization

The reported accuracy is based on the available test dataset and may differ when the model is applied to completely different datasets or real-world handwriting.

14. FUTURE ENHANCEMENTS

The project can be extended in several ways.

1. Word and Sentence Recognition

The system could be extended to recognize complete words and sentences.

2. Larger and More Diverse Dataset

More handwriting samples could be collected from different writing styles.

3. Improved CNN Architecture

More advanced architectures could be explored to improve performance.

4. Data Augmentation

Additional controlled transformations could improve model robustness.

5. Web Application Deployment

The model could be deployed through a web application.

6. Mobile Application

The model could potentially be integrated into a mobile application.

7. Real-Time Recognition

Camera-based handwritten character recognition could be explored.

8. Additional Character Classes

The system could be extended to recognize digits and special characters.

9. Model Optimization

Quantization and other optimization techniques could reduce model size and improve inference speed.

10. Enhanced GUI

Prediction history, confidence charts, and additional interface features could be added.

15. CHALLENGES FACED

During development of the Handwritten Alphabet Detector, several challenges were addressed.

15.1 Dataset Organization

The handwritten character dataset contained multiple directories and class labels. Correctly organizing the images and associating each image with the appropriate alphabet label was necessary for successful training.

15.2 Image Preprocessing

Images may differ in size, position, background, and writing style. Developing a consistent preprocessing pipeline was therefore important.

15.3 Character Alignment

Characters may not be positioned at the center of an input image. Character detection, cropping, and padding were used to create a more consistent representation.

15.4 Model Generalization

A high training accuracy does not necessarily guarantee equivalent performance on unseen images. Validation and test datasets were therefore used to evaluate generalization.

15.5 GUI and Model Integration

Integrating the trained CNN with a Tkinter application required the same preprocessing logic to be applied to newly drawn or uploaded images.

15.6 Prediction Consistency

The input representation used during GUI prediction must be compatible with the representation used during model training. Maintaining consistent image dimensions, grayscale processing, normalization, and orientation was therefore important.

16. CONCLUSION

The Handwritten Alphabet Detector was successfully developed using a Convolutional Neural Network and the TensorFlow/Keras deep learning framework.

The project demonstrates a complete image-classification workflow beginning with dataset preparation and image preprocessing and continuing through CNN development, model training, evaluation, prediction, model saving, and GUI implementation.

The handwritten alphabet images were processed as 28×28 grayscale images. The CNN learned important visual features such as edges, curves, shapes, and patterns and classified the images into 26 alphabet classes from A to Z.

The reported model achieved a test accuracy of 96.15% on 286 test images, with a reported test loss of 0.1627. Additional evaluation was performed using a classification report and confusion matrix.

A Tkinter-based graphical user interface was developed to make the system interactive. Users can draw an alphabet using a mouse or upload an existing image. The application preprocesses the input, loads the trained CNN model, and displays the predicted alphabet, confidence score, and top prediction results.

During individual prediction testing, the sample image A_caps.004.png was correctly classified as A with 99.82% confidence.

Overall, the project demonstrates the practical application of deep learning, computer vision, image preprocessing, CNN-based classification, model evaluation, and graphical user interface development to handwritten alphabet recognition.

The developed system provides a foundation for future extensions such as word recognition, real-time recognition, web deployment, mobile applications, and recognition of additional character classes.

17. REFERENCES

1. TensorFlow Documentation – TensorFlow and Keras documentation.
2. Keras Documentation – Deep learning model development and training.
3. Scikit-learn Documentation – Classification metrics and model evaluation.
4. Python Documentation – Python programming language documentation.
5. Pillow Documentation – Python Imaging Library documentation.
6. Matplotlib Documentation – Data visualization and plotting.
7. Tkinter Documentation – Python graphical user interface development.
8. Kaggle – Handwritten English Characters and Digits Dataset.
9. Course materials and lecture notes on Convolutional Neural Networks and Deep Learning.